Journal of King Saud University – Computer and Information Sciences 35 (2023) 101710

HOSTED BY

Contents lists available at ScienceDirect

Journal of King Saud University – Computer and Information Sciences

journal homepage: www.sciencedirect.com

Reputation aware optimal team formation for collaborative software crowdsourcing in industry 5.0

Syeda Nabila Akter [a], Afsana Kabir Sinthia [a], Palash Roy [a], Md. Abdur Razzaque [b], Mohammad Mehedi Hassan [c,*], Francesco Pupo [d], Giancarlo Fortino [d]

[a] Department of Computer Science and Engineering, Green University of Bangladesh, Dhaka, Bangladesh
[b] Green Networking Research Group, Department of Computer Science and Engineering, University of Dhaka, Bangladesh
[c] Information Systems Department, College of Computer and Information Sciences, King Saud University, Riyadh 11543, Saudi Arabia
[d] Department of Informatics, Modeling, Electronics, and Systems, University of Calabria, Rende 87036, Italy

ARTICLE INFO

Article history:
Received 1 April 2023
Revised 13 July 2023
Accepted 6 August 2023
Available online 12 August 2023

Keywords:
Collaborative software crowdsourcing
Fair worker selection
Particle swarm optimization
Industry 5.0
Quality-of-experience

ABSTRACT

Collaborative software crowdsourcing (CSC) is now a vital part of the technological workforce due to the rising need for software solutions in an industry 5.0 smart city that exceeds performance standards without sacrificing cost-effectiveness. When a buyer intends to hire a group of productive workers within their budget limit and time frame, the CSC offers a potential solution for executing a software module in crowdsourcing. Existing literature works are limited by team-building mechanisms in CSC without considering reputation and technical competency, which are crucial to building an effective team. In this paper, we have developed an AI-based meta-heuristic particle swarm optimization (PSO) algorithm, which gives personalized solutions for intelligent CSC team-building problems, namely PCS system, that effectively forms worker-teams. We furthermore develop a reputation-based scheme by considering workers' competency and prior performance to enhance the dependability and trustworthiness of the workers in the CSC platform by employing an exponentially weighted moving average (EWMA) and the multi-step time balancing weight-based method. The developed PCS system also offers a utility function to buyers weighing the cost against task quality. All experiments are carried out in MATLAB and the results have shown that the developed PCS system enhances the buyer's quality-of-experience (QoE) and utility as high as 30% and 25%, respectively compared to the state-of-the-art works.
© 2023 The Author(s). Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

1. Introduction

Crowdsourcing refers to individuals, either alone or in groups, contributing to a project through a collaborative platform with a mutual agreement to influence social resources from a vast network of contributors (Huq et al., 2019). The platform, the requester or buyer, and the worker are the three main entities of a crowdsourcing system. These three parties are linked via projects that can be finished by a group of people or individual people (Wang et al., 2017a; Wang et al., 2019; Jiang et al., 2022). Crowdsourcing is often employed in order to fulfill jobs like website usability assessment, microdata gathering, and other similar ones, and these tasks are completed via a crowdsourcing platform. Amazon Mechanical Turk is one example of a microtask-based crowdsourcing platform (Jiang et al., 2020). Here, more than 4,00,000 registered crowd workers are connected to accomplish their demanded tasks. Additionally, Fiverr, Freelancer, and Upwork are some renowned software crowdsourcing platforms, and these global online marketplaces provide works that are more complex and skill-dependent, e.g., web development, UX/UI designing, technical writing, and so on, which help to modernize the traditional work system and eventually help in developing smart cities (Wagner et al., 2021; Machado et al., 2021; Souza-Pereira et al., 2022; Sorour et al., 2022).

Online marketplaces and collaborative crowdsourcing workplaces exemplify Industry 5.0 systems by leveraging automation, and personalized experiences according to the buyer's requirement

* Corresponding author.
E-mail addresses: razzaque@du.ac.bd (M.A. Razzaque), mmhassan@Ksu.edu.sa (M.M. Hassan), francesco.pupo@unical.it (F. Pupo), giancarlo.fortino@unical.it (G. Fortino).

Peer review under responsibility of King Saud University.

Production and hosting by Elsevier

https://doi.org/10.1016/j.jksuci.2023.101710
1319-1578/© 2023 The Author(s). Published by Elsevier B.V. on behalf of King Saud University.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

(E. Commission et al., 2021; Maddikunta et al., 2022). They enable balanced human–machine collaborations and remote team collaboration and enhance resilience during disruptions such as natural disasters, pandemics, or other unexpected events (Chakir et al., 2023). Software crowdsourcing in a collaborative environment fosters sustainability through knowledge and skill sharing, which in turn enhances motivation and engagement among participants. It promotes the development of sustainable solutions by leveraging diverse perspectives for solving complex problems, reducing the need for physical infrastructure and travel, and lowering carbon emissions and energy consumption.

In software crowdsourcing, the requester or buyer hires workers for a software project within a specific domain and timeframe, with or without a designated leader (Yu et al., 2019; Stol et al., 2017; Haqbani and Alyahya, 2022). An example scenario of a crowdsourcing environment is depicted in Fig. 1, where buyers from different corners of the world post tasks on crowdsourcing platforms while workers from across the world bid to complete them. Workers complete tasks and return the results back to the buyer. Then the buyer reviews the tasks and sends payment and perhaps a reward. If truthful workers are not chosen, then selfish workers can take advantage of the crowd sourced market by behaving dishonestly (Wang et al., 2019; Li et al., 2015; Sun et al., 2022). As a result, other workers are dissuaded from taking part, which prevents the CSC platform from discovering the best workers to increase the quality of the crowdsourcing platform. Therefore, determining a worker's reputation accurately has become a major challenging task.

Mobility is essential for building a smart city, and AI can make tasks easier, safer, and more productive by performing ordinary and often even difficult tasks more effectively than humans can. The social breakdown caused by unemployment during the COVID-19 pandemic is a challenging scenario for our work. To ensure financial gains, there needs to be a better crowdsourcing platform that distributes jobs to suitable individuals while offering reasonable cost-quality trade-offs (Wang et al., 2019; Huang and Chen, 2019). It is also essential to automate all tedious work in crowdsourcing, like worker selection by the platform, which helps humans (i.e., buyers and workers) to use their natural intellect more efficiently, which is very necessary for Industry 5.0. Additionally, there needs to be an efficient reputation mechanism that matches both the institutional and field-specific experience of the workers (Huang and Chen, 2019). To automatically match tasks and developers, Mao et al. (2015) have suggested a content-based recommendation strategy that uses previous data to train a classification model and assign new tasks based on task similarity. This method relies on the validity of historical data and ignores workers' collaboration. Wang et al. (2017b) have developed a capability improvement model, which proposes the recommendation value of the crowdsourcing software developers. However, the authors have not provided any solution to the problem of software module resolution in a team environment.

These findings have led us to conclude that an appropriate reputation scheme can assess a worker's abilities and detect dishonest behavior, guaranteeing the caliber of work and the platform's effectiveness. This paper exploits crowdsourcing as a solution to detect untruthful worker behavior and addresses the intelligent and personalized team-building mechanism while working on collaborative tasks. In contrast to a conventional software development environment, large-scale software engineering on crowdsourcing platforms faces various issues, one of which is figuring out how to split, schedule, and integrate projects while maintaining overall quality and productivity.

A preliminary version of this work addressed a basic reputation scheme for worker selection in software crowdsourcing (Rahman et al., 2021). In this paper, we fine-tuned the reputation scheme by incorporating additional contributing factors in a worker's competency and performance. We have also developed a **P**article

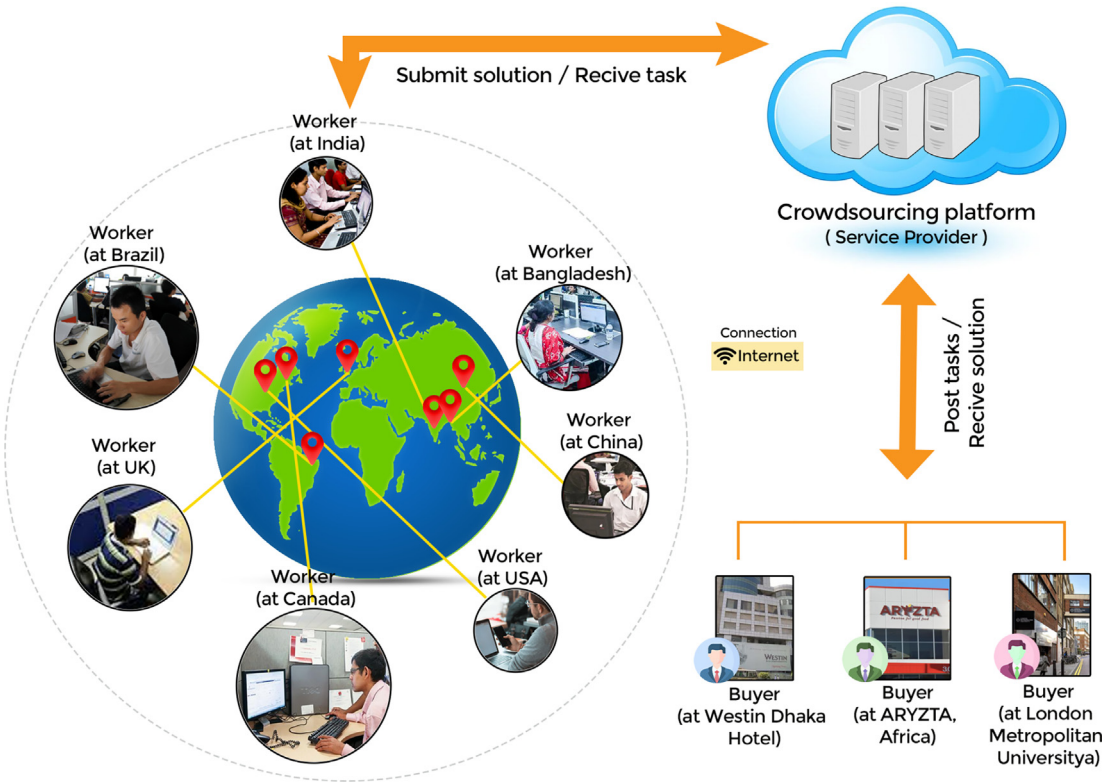


Fig. 1. An example of a typical software crowdsourcing system.

swarm optimization based **C**ollaborative **S**oftware crowdsourcing, namely PCS, that can successfully discover and form an optimal team from multiple pools of filtered workers, as well as provide a trustworthy platform for software crowdsourcing. Our work complements the "Industry 5.0" approach through automated worker selection, worker-team building approach, and payment-reward calculation, which leads to a resilient system where workers can focus more on their actual tasks. The PCS system assures that the team is formed not by considering a computationally expensive greedy algorithm; it conveys a meta-heuristic AI-based particle swarm optimization (PSO) technique instead. The main contribution of this study can be summed up as follows:

- We have developed an enhanced reputation scheme in a collaborative crowdsourcing environment that influences worker's technical proficiency, soft skills, and performance.
- We have devised a meta-heuristic AI-based PSO team formation mechanism, where the buyer has the freedom to set a measurement for their desired team quality, and a truthful worker has the advantage of getting an incentive according to task quality.
- We have established a payment policy for selected workers where both buyers and workers will benefit as workers will get a reward or penalty as per their task quality.
- We implement and evaluate our proposed PCS mechanism in MATLAB and compare its performances with those of other state-of-the-art works.

The remaining paper is organized as follows. Section 2 describes related literature works on the field of Software crowdsourcing. The developed system model and assumptions are described in Section 3 and subsequently Section 4 describes the detailed reputation calculated scheme and particle swarm optimization-based worker selection mechanism. Results of the comparative performance analysis are shown in Section 5 and The overview of the entire paper and recommendations for additional research are given in Section 6.

2. Related work

With the advancement of crowdsourcing technology and the growth of the population, collaborative remote work can improve job efficiency while also utilizing the population for smart living and smart city development. Traditional crowdsourcing frequently concentrates on finishing small tasks that any worker can accomplish on their own (Wang et al., 2019; Li et al., 2015; Roy et al., 2021; Sarker et al., 2019; Shen et al., 2022; Rahman et al., 2021). However, as crowdsourcing activities evolve, it is now crucial to develop a human-centric, resilient system to recruit a group of competent and socially connected professionals that incorporates strong honesty from both the buyer and worker sides (Huang and Chen, 2019; Liao et al., 2021; Dubey et al., 2016), that promotes human energy sustainability per the Industry 5.0.

In Zhang et al. (2018); Li et al., 2015, the authors have developed a task distribution framework in crowdsourcing, which is treated as a non-manipulated reverse auction. Single-unit multi-attribute auction-based mechanism (SUMA) and multi-unit multi-attribute auction (MUMA) auctions ensure the truthful value of the buyer's bidding strategies and maximize social utility, which is suggested in Xiao et al. (2021), giving a strong theoretical foundation for operating a long-term reliable crowd distribution system. Multiple auctions were utilized to ensure accuracy and a worthwhile outcome. They have utilized a predetermined payment system that only considers the buyer's side but ignores the worker's side. In Huang and Chen (2019), the authors have suggested a reputation evaluation methodology based on linear discriminant analysis (LDA) for the users of Random Forest (RF) crowdsourcing. The findings show that their model LDA-RF outperforms competing models in terms of performance and efficacy as well as accuracy, F1-measure, generalization ability, and durability. This study offers a novel perspective on assessing the reputation of crowdsourcing participants in a big data environment.

Roy et al. (2021) have developed a multi-weight subjective logic (MWSL) based reputation method by taking into account the interaction effects, freshness, similarity, and frequency, and in the evaluation of workers' reputations, the importance is placed on individual worker's communication skills. However, they have not taken into account the workers' technical proficiency in the assessment of workers' reputations, which is a fundamental component of software crowdsourcing. Another group of authors in Liao et al. (2021) have developed an algorithm namely GRBMC, that mines the reputation and activity characteristics of workers based on the worker ability matrix and historical data and then classifies the reputation community, the activity community, and the preference community among crowdsourcing workers using classification and clustering algorithms. They have calculated the reputation based on the worker ability matrix and historical data, which is not sufficient enough to measure worker stature correctly. The preliminary version of our current work was limited to introducing a worker selection method following a basic reputation scheme (Rahman et al., 2021), where a deliberated consideration is crucial to select a worker in the collaborative software crowdsourcing environment. To make this system more resilient and sustainable (E. Commission et al., 2021), an advanced crowdsourcing approach is needed so that workers can work in a collaborative manner, which had been incorporated and implemented into our current system by ensuring fair payment to workers.

In Wang et al. (2022); Gutiérrez et al., 2016, the authors introduced a heuristic person-job matching algorithm (HPJMA) to assess job applicants' suitability based on specific requirements. However, this algorithm only considers a limited number of factors and does not incorporate a reputation scheme for team formation, which is crucial for software projects. In Wang et al. (2022); Guo et al., 2023, the authors proposed genetic algorithm-based solutions in different contexts, but these approaches fail to meet the requirements for crowdsourcing team formation on a large scale. For software crowdsourcing environments, it is essential to have exploration and diverse search capabilities to effectively build extensive worker teams. One of the most important concerns in the development of software crowdsourcing has always been the way in which it maintains worker collaboration.

A work assignment system based on user reliability has been proposed in Mao et al. (2015). This mechanism accounts for differences in worker reliability but ignores task difficulty differences. On the other hand, in Fu and Song (2015), the authors put out the notion of forming collaborative working groups based on the coincidence of active time. Considering that, it has been suggested to solve the task assignment problem of total utility optimization under the cooperative constraint. The groups are generated based on the worker's active times, but they have not taken into account communication between workers within a team. Wang et al. (2017b) have suggested modeling skill growth with learning curve models and confirmed that the negative exponential learning curve was appropriate for describing the skill improvement of developers in Topcoder. They have used a combination of neural networks and content-based filtering to recommend a developer. It is based on intrinsic properties but ignores the dynamic development capabilities of developers. In Yu et al. (2019), the authors have proposed a

solution for multi-task to multi-worker assignments and handled collaboration in software crowdsourcing, namely STAM. However, it ignores the value of workers and platform profit. In order to give the task to the group's workers, they have separated the task into sub-modules. However, the authors have not taken into account the timeliness and communication between workers for allocating the task. Meta-heuristic strategy for optimal worker selections is a better approach, according to several studies (Hamrouni et al., 2022; Wang et al., 2021; Shen et al., 2022; Heling et al., 2023). However, only a few papers have addressed the use of meta-heuristics in crowdsourcing.

Most existing works focused on resolving micro-level tasks rather than solving a real-world software development problem. A few studies that addressed solving software development problems had a lack of thinking in reliable worker selection and effective worker-team formation mechanism. Moreover, none of these previous studies has focused on reputation-based reverse auction techniques in the context of software crowdsourcing. In a software crowdsourcing environment, minimizing the cost of tasks and offering lucrative payments to workers based on the quality of their task performance is a complicated issue. Based on these observations, we have developed a meta-heuristic AI-based PSO optimal team formation mechanism in collaborative software crowdsourcing by considering the worker's technical competency and prior performances. The iterative nature of the meta-heuristic algorithm gives a better solution for the effective team-finding problem and offers a faster convergence speed for the system.

3. System model and assumptions

We are considering a crowdsourcing platform consisting of three entities workers (W), buyers (B), and the CSC platform as depicted in Fig. 2. This is a centralized system where buyers have large-scale software modules required for software crowdsourcing with the support of a professional worker team. The platform works as a broker and helps to form an optimal team for addressing a software development problem focusing on key goals of Industry 5.0.

Let, B denotes a set of buyers, where each buyer is referred by $b_j \in B$. Each buyer $b_j \in B$ is responsible for submitting one software module, $\delta_i \in \Delta$. The δ_i denotes a set of tasks in a given software module. Numerous sub-modules may exist inside each software module. For example, an e-commerce application is a software module and its UI/UX design is a sub-module. The worker set is denoted by W and the index of each worker is $w_i \in W$. The bidding attribute of each worker $w_i \in W$ can be represented using four attribute tuples $< \mathscr{C}, \mathscr{P}, c, t >$, where, $\mathscr{C}$ represents the worker's technical competency that is measured by worker's technical and soft skill set, $\mathscr{P}$ denotes the worker's previous performance that can be both inside and outside the platform, c is a monetary cost and t indicates the worker's availability time. In the developed PCS system, workers' reputation value is calculated based on the worker's technical competency and previous performances. On the other hand, buyers only work together to evaluate the worker's reputations and find the most desired team. Let, $\mathbb{T}$ be the desired

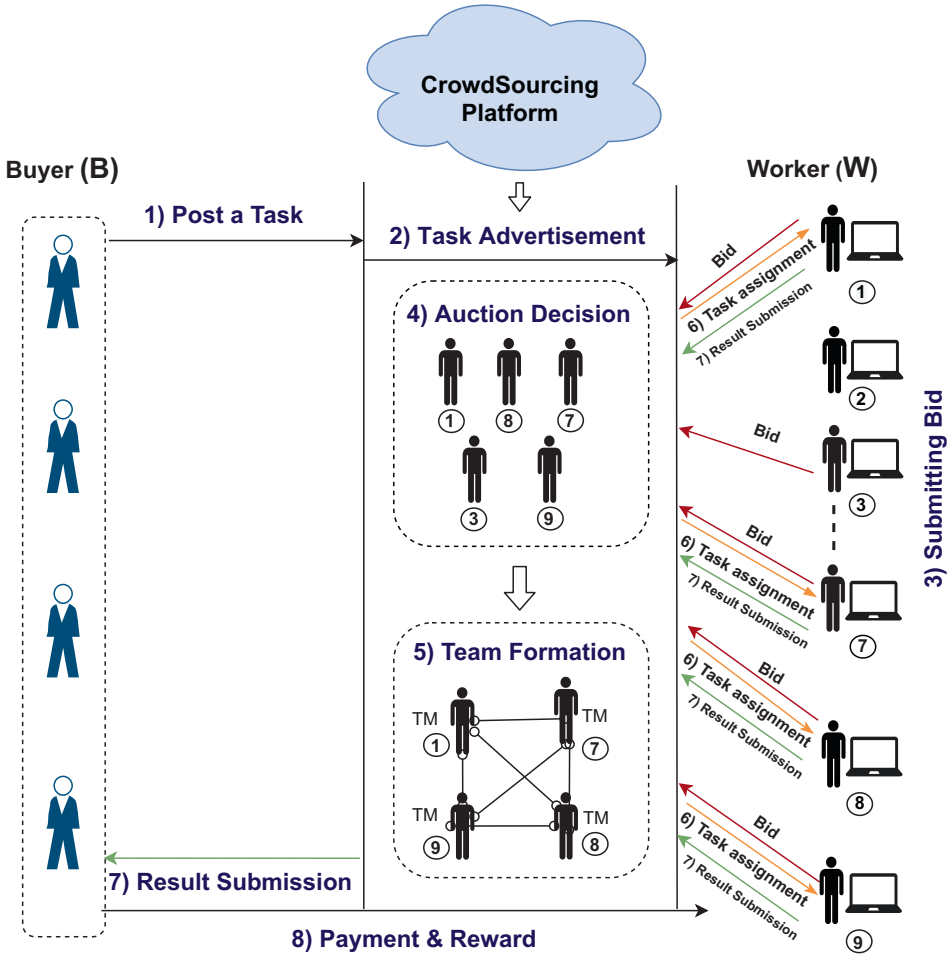


Fig. 2. Interactions among the entities in a collaborative software crowdsourcing system

crowdsourcing team, that contains the optimal set of workers within the required skill and cost from the available workers.

In PCS, a worker $w_i \in W$ gets the claimed price after the successful completion of the tasks. Along with the standard compensation, the worker may also receive an incentive or reward if the task is completed well by the worker. Here, incentive refers to the monetary value added to the worker's claimed cost in order to generate payment ρ. As a result, the final payment is equal to the sum of the workers' claimed costs plus the incentive amount. On the other hand, a worker gets punished if he or she completes a poor-quality task. In that instance, the worker is paid less than the stated amount. The detailed step-by-step exchanges between the buyer and the worker in our developed PCS setting, depicted in Fig. 2 are listed as follows.

Step 1: The buyer will first post a task and the skills required for it on the crowdsourcing platform.
Step 2: The platform will receive the posted task notification from the buyers and advertise the tasks to the workers.
Step 3: In this step, the workers will bid in order to be selected within a time frame t.
Step 4: On a platform, reverse auction decisions will be made based on the workers' bids. Workers will be short-listed based on their technical competency $\mathscr{C}_i$ and performance $\mathscr{P}_i$.
Step 5: The platform will then select the best set of workers depending on the buyer's requirements and then optimal software crowdsourcing team $\mathbb{T}$ will be formed.
Step 6: After forming a team, workers will be notified and the allocated work will be given to the team.
Step 7: After the task is completed, the selected team $\mathbb{T}$ gives the platform their end outcome and the platform sends the results to the buyers.
Step 8: When a task is successfully finished, the buyer evaluates the performance standard of the workers and gives them rewards φ in addition to their normal compensation ρ_i , and provides feedback to each worker $w_i \in W$ based on their contributions. The major notations used in this paper are listed in Table 1.

Table 1
List of notations.

Symbols	Description
B	Set of buyers
W	Set of workers
Δ	Set of tasks in a given software module
S	Candidate worker set
c_i	The claimed cost of worker $w_i \in W$
Ω_i	Communication proficiency of worker $w_i \in W$
$\mathscr{C}_i$	Technical competency of worker $w_i \in W$
T_j^d	Task delay deadline of buyer $b_j \in B$
$\lambda_{n,i}^t$	Internal reviews of worker w_i after n^{th} project done through platform at time t
$e_{n,i}^t$	External reviews of worker w_i after n^{th} project done outside platform at time t
Ω_n^m	Communication proficiency of team work capacity m after completing n^{th} project
$\mathscr{T}_i$	Timeliness of worker $w_i \in W$ after n^{th} project
$\mathscr{P}_i$	Performance of worker $w_i \in W$
$\mathfrak{L}_i^p$	Local best of the ith component of particle $p \in P$
$\mathfrak{g}_{best}$	Global best of the ith component of particle $p \in P$
ρ_i	Payment of worker $w_i \in W$
$\mathbb{T}$	Desired team
q_i	Task quality of worker $w_i \in W$
$\mathscr{U}_j$	Utility of buyer $b_j \in B$

4. Design details of PCS system

In this section, we have developed a reputation scheme considering possible contributing factors in a worker's competency and performance to maintain maximum fairness in worker selection. We have also developed a meta-heuristic Particle Swarm Optimization (PSO) based team formation mechanism. And finally, we have proposed a worker payment policy.

4.1. Computation of worker reputation

Worker reputation plays a vital role in selecting the most reliable workers. Low-quality workers may be picked while their genuine quality is hidden caused by the fact that the reputation computation mechanism isn't tight and fair. In the proposed PCS system, the reputation calculation methodology is designed to encompass two key parameters: the worker's competency and prior performance. To calculate these parameters, we further break them into several sub-parameters, as described below, to deduce a comprehensive measurement of the quantified values.

4.1.1. Worker's competency

The most important reputation parameter for a worker is his or her competency, which is affected by several factors including technical expertise and diversity, and communication proficiency in a team (Tommaselli et al., 2022), described below.

- **Expertise and Diversity:** The expertise and diversity define how much experience and diversity of knowledge a worker possesses in the related work domain (e.g., programming language, database design, prototype design, and so on). Different software development tasks require various technical competencies, and by having workers with diverse expertise, the overall capacity to handle different types of projects is increased. Here, the experience and diversity of worker w_i is denoted by Υ_i. Using Simpsons's Diversity Index (Routledge, 1979), which is a diversity indicator that considers both the total number of the complete project and the relative abundance of each project of a worker $w_i \in W$ in their working domain among the whole worker set W, can be calculated as follows,

$$\Upsilon_i = 1 - \frac{\sum_{l=1}^{v} \xi_i^l(\xi_i^l - 1)}{N_i(N_i - 1)}, \tag{1}$$

where, N_i is the total number of projects done by worker $w_i \in W$, v denotes total number of working domains and ξ_i^l indicates the number of projects done in that particular domain l by worker w_i. This equation depicts that the more experience a worker has working in diverse areas, the higher rating he or she will get.

- **Worker's communication proficiency:** A worker's communication proficiency may also include truthfulness, verification of education, fluency in a human language, such as English, behavior, mutual respect, or other platform-specific standards for human communication. The exponentially weighted moving average (EWMA) is used to determine the value of individual worker's communication proficiency after completing n projects on the platform,

$$\Omega_i = \begin{cases} \mathbb{C}_i^0 & n = 0 \\ \alpha \times \mathbb{C}_i^n + (1 - \alpha) \times \Omega_i^{n-1} & n > 0, \end{cases} \tag{2}$$

where, the weight factor for the most recent value of $\mathbb{C}_i^n$ is α $(0 \leqslant \alpha \leq 1)$. The platform provides the initial communication

proficiency value $\mathbb{C}_i^0$ established on the platform's developed evaluation, and the platform sets the initial value of Ω_i to zero for the communication proficiency value and the value of Ω_i is updated after each project is completed.

- **Worker's teamwork capacity:** While technical ability and individual communication skills are very important for selecting a worker, our developed PCS platform also values each worker's team communication skills to improve the effectiveness of the team as a whole. This parameter helps to filter effective workers in the team environment of the proposed system. The communication proficiency Ω_n^m of each worker's teamwork capacity m after completing n^{th} project can be calculated using Eq. (3) as follows,

$$\Omega_m^n = \begin{cases} \mathbb{C}_m^0 & n = 0 \\ \alpha_m \times \mathbb{C}_m^n + (1-\alpha_m) \times \Omega_m^{n-1} & n > 0. \end{cases} \tag{3}$$

Here, α_m $(0 \leqslant \alpha_m \leq 1)$ is the weight factor, which makes a balance between the recent and past values of a worker's teamwork capacity.

Now, we can calculate the overall competency $\mathscr{C}_i$ of a worker $w_i \in W$ based on the weighted sum of the worker's expertise and diversity, Υ_i and communication proficiency, Ω_i. Here, $\Upsilon_i, \Omega_i \in [0,1]$ and $\Upsilon_i + \Omega_i = 1$. The final competency value $\mathscr{C}_i$ of worker $w_i \in W$ competency can be calculated using Eq. (4) as follows,

$$\mathscr{C}_i = \zeta \times \Upsilon_i + (1-\zeta) \times \Omega_i, \tag{4}$$

where, ζ is a weight parameter ranges in $[0,1]$, which is set by individual buyers based on their own choices.

4.1.2. Worker performance

A worker's previous performance history plays an influential role along with a worker's competency to measure the overall worker's reputation. By analyzing workers' previous performance history, it is possible to determine workers' knowledge and expertise in a certain domain. In this section, each worker's prior performance is calculated based on their project reviews, re-employment status, and timeliness.

- **Project Review (β):** This parameter considers various factors such as the employee's project's reputation and market value, the popularity of the software products they contribute to, and their past experiences both within and outside the platform. Previous performance history provides insight into the quality of the worker's work, client satisfaction levels, and overall performance on past projects. The EWMA technique has been exploited to rate the worker's project so that we can emphasize more on workers' recent works than the past ones while evaluating them. The worker's external and internal reviews for an individual project denoted by $\mathbb{E}_i^t$ and $\mathbb{I}_i^t$ have a higher impact on averaging the ratings that produce more truthfulness because they place more weight on the worker's recent work. Because workers can engage honestly in previous projects while showing increasingly unsatisfactory results over time. The value of external and internal reviews per project, $\mathbb{E}_i^t$ and $\mathbb{I}_i^t$, respectively, for the review of worker $w_i \in W$ at time slot t can be determined as follows:

$$\mathbb{E}_{1,i}^t = \begin{cases} E_{1,i}^t, & t = 1, \\ \kappa_e \times E_i^t + (1-\kappa_e) \times \mathbb{E}_{1,i}^{t-1}, & t > 1, \kappa_e > (1-\kappa_e). \end{cases} \tag{5}$$

$$\mathbb{I}_{1,i}^t = \begin{cases} I_{1,i}^t, & t = 1, \\ \kappa_x \times I_i^t + (1-\kappa_x) \times \mathbb{I}_{1,i}^{t-1}, & t > 1, \kappa_x > (1-\kappa_x). \end{cases} \tag{6}$$

Here, κ_e and κ_x are two weighting factors ranges in [0,1], those make a trade-off between the recent and past time external and internal project reviews for an individual project, respectively. Now, we can calculate the overall external and internal project reviews by Eqs. (7) and (8) as follows,

$$\lambda_{n,i}^t = \begin{cases} \mathbb{I}_{1,i}^t & n = 1 \\ \alpha_x \times \mathbb{I}_{n,i}^t + (1-\alpha_x) \times \lambda_{n-1}^t \end{cases} \tag{7}$$

$$e_{n,i}^t = \begin{cases} \mathbb{E}_{1,i}^t & n = 1 \\ \alpha_e \times \mathbb{E}_{n,i}^t + (1-\alpha_e) \times e_{n-1}^t \end{cases} \tag{8}$$

Here, α_x $(0 \leqslant \alpha_x \leq 1)$ and α_e $(0 \leqslant \alpha_e \leq 1)$ are the weight factors and $\alpha_x < (1-\alpha_x)$ and $\alpha_e < (1-\alpha_e)$. The opinions of older projects carry more weight as they have had a longer duration to accumulate reviews, whereas newer projects may not have garnered enough reviews yet to be deemed fairly justified. Now, the worker's technical competency has been estimated by weighted summation of both internal and external project's EWMA value at time t using Eq. (9) as follows,

$$\beta_i = \psi_y \times \lambda_{n,i}^t + \psi_x \times e_{n,i}^t. \tag{9}$$

Here, $\psi_y + \psi_x = 1$. Weights for external projects ψ_x and internal projects ψ_y can be determined by the buyer within a range of 0 to 1. By default, ψ_y will be greater than ψ_x, which indicates projects inside the platform will be given more weight in general.

- **Worker's Reemployment Status (μ):** This rating point denotes the number of times a worker has been rehired as well as whether or not he has been rehired by a single buyer or multiple buyers. if a worker is rehired by the same buyer and does not work with any other buyers for a noticeably longer period then the worker's reemployment status rating μ_i will be lower, and it can be inferred that there may have some mutual benefit in between them, which may violate the platform policy to maintain truthfulness. This way, a higher reemployment rating will ensure the truthful behavior of a worker more prominently. Worker's reemployment status rating μ_i can be calculated using Eq. (10) as follows,

$$\mu_i = 1 - \frac{\sum_{b_j \in B} \phi_{i,j}^r (\phi_{i,j}^r - 1)}{N_i^r (N_i^r - 1)}, \tag{10}$$

where, $\phi_{i,j}^r$ means the number of times worker w_i is re-employed by the same buyer $b_j \in B$, and N_i^r means the total number of times employed.

- **Timeliness:**This rating attribute in worker performance signifies how punctual a worker is in successfully submitting the assigned task. The timeliness value $\mathscr{T}_i$ of worker $w_i \in W$ can be calculated as follows,

$$\mathscr{T}_i = 1 - \frac{\sum_{n=1}^{n} \left(\frac{t_{n,i}}{T_j^d} \right)}{n}, \tag{11}$$

where, T_j^d is the task delay deadline of the application and $t_{n,i}$ is the worker's task completion time of project n. Here, In Eq. (11), the task completion time of a worker is always less or equal to the task delay deadline, i.e., $t_{n,i} \leqslant T_j^d$. Here, timeliness $\mathscr{T} \in$ [0,1]. The lower timeliness value $\mathscr{T}$, indicates the higher reputation value of the workers.

Now, we can calculate the final value of the previous performance $\mathscr{P}_i$ of worker $w_i \in W$ from the weighted sum of project review, β, worker's reemployment status μ and timeliness $\mathscr{T}$ as follows,

$$\mathscr{P}_i = \eta_1 \times \beta_i + \eta_2 \times \mu_i + \eta_3 \times \mathscr{T}_i, \tag{12}$$

where, $\beta_i, \mu_i, \mathscr{T}_i \in [0,1]$. η_1, η_2, η_3 are the weight factors, where, $\eta_1 + \eta_2 + \eta_3 = 1$ and these values are set by the buyers individually based on their own necessity. Therefore, the final value of worker's performance $\mathscr{P}_i$ ranges in $[0,1]$.

4.2. PCS reverse auction and worker filtration

In this section, we will describe the developed PCS system's reverse auction mechanism and worker filtration process.

In Algorithm 1, the PCS uses reverse auction and a worker filtering process happens. The buyers initially submit input values $\mathscr{P}, \mathscr{C}, t$, and c for the task and a filtering list of workers $S \subseteq W$ is prepared those satisfy the following constraints:

1. *Worker's performance constraint*: Worker performance must exceed the minimum requirements, $\mathscr{P}\prime$, i.e., $\mathscr{P}_i \geqslant \mathscr{P}\prime$.
2. *Worker's competency constraint*: Worker technical competency must be higher than the minimum competency requirements, $\mathscr{C}\prime$, i.e., $\mathscr{C}_i \geqslant \mathscr{C}\prime$.
3. *Deadline constraint*: Task delay deadline T_j^d of buyer b_j must be greater than or equal to the execution time t_i of worker w_i. Here, T_j^d is provided by the buyer and t_i is calculated by the platform.
4. *Budget constraint*: The worker's claimed cost c_i must be less than or equal to the maximum affordable cost or budget of the buyer's C_j'' that the buyer b_j has provided.

If no candidate worker $s_i \in S$ gets selected according to the buyer's reputation threshold, parameter re-initialization happens in line 4, i.e., further relaxation may be needed while setting the threshold value of the parameters. In the initial pool, only the workers who satisfy all four requirements will be shortlisted. In line 6, either time and cost will be sacrificed in order to enhance worker's performance $\mathscr{P}$ and competency $\mathscr{C}$ suited for the situation or vice versa. Here, the advancement choice $a \in A$ is a buyer's input that let him select the workers based on his needs. If the choice of a is $\mathscr{P}$ or $\mathscr{C}$, then time and cost will be sacrificed in order to enhance the quality. However, the quality of the worker will be degraded to reduce time and cost if a is chosen over t or c.

Algorithm 1 Worker Filtering Algorithm

Input: Worker's minimum performance $\mathscr{P}'$, minimum Competency $\mathscr{C}'$, Task delay deadline T_j^d, Maximum affordable cost C_j'', Advancement choice a.
Output: Candidate Worker Set S
1: $S' \leftarrow \varnothing$
2: $S' = \{w_i | \mathscr{P}_i \geqslant \mathscr{P}', \mathscr{C}_i \geqslant \mathscr{C}', t_i \leqslant T_j^d, c_i \leqslant C_j'', \forall w_i \in W\}$
3: **if** $|S'| == 0$ **then**
4: Reinitialize the parameter values of $\mathscr{P}', \mathscr{C}', T_j^d, C_j''$
5: **go to** step 2
6: **else**

$$S = \begin{cases} s | \underset{a \in A}{\operatorname{argmin}}(a), \forall s \in S \text{ if } a = \mathscr{P} || a = \mathscr{C} \\ s | \underset{a \in A}{\operatorname{argmin}}(a), \forall s \in S \text{ if } a = t || a = c \end{cases}$$

7: **end if**
8: **return** S

4.3. Optimal PCS team formation

After filtering the candidate workers using Algorithm 1, we have developed a meta-heuristic particle swarm optimization (PSO) based team formation mechanism. The main advantage of employing a meta-heuristic algorithm over heuristic, greedy, or optimization methods is that they enable us to discover a feasible solution to a problem with less computational effort. In contrast to heuristic algorithms which are tailored to specific problems, meta-heuristic solutions are not problem-dependent. Although they may not guarantee the optimal solution, they can efficiently find a solution that is close to the optimal one within a reasonable time frame (Gu et al., 2022). Furthermore, the decision to utilize PSO as the meta-heuristic algorithm for team formation in the PCS system may have been influenced by the specific advantages that PSO offers in this particular context. In comparison to alternative meta-heuristic algorithms like a genetic algorithm (GA), PSO has demonstrated superior efficiency in terms of faster convergence speed (Duan et al., 2009). The PSO is a population-based meta-heuristic algorithm inspired by the movement of a flock of birds or a school of fish. Particles are the smallest units of the population or swarms in PSO. To search for a solution, particles move through a multi-dimensional search space by controlling their position and velocity. In each iteration, each particle aims to improve the solution of a fitness function. The particle's movement is influenced by its personal best position found so far and the overall best position of the entire swarm in the search space, which helps them progress towards better solutions (Roy et al., 2020; Estrada et al., 2022).

Algorithm 2 Optimal Team Formation Algorithm

Input: Candidate Worker Set S, Total number of members in a team required $\mathscr{N}$, Maximum number of iteration $MAX - IT$
Output: Desired Team $\mathbb{T}$
1: Initialize system Parameters $z_1, z_2, \mathbb{T} \leftarrow \varnothing$
2: Set maximum iteration MAX-IT
3: **for all** $p \in P$ **do**
4: Generate initial position $x^p \sim U(b_l, b_u)$ of position vector X^p
5: Initialize velocity, $v^p \sim U(-|b_l, b_u|, |b_l, b_u|)$
6: **end for**
7: **while** $(Itr \leqslant MAX - IT)$ **do**
8: **for all** particle $p \in P$ **do**
9: **for all** dimension $x_i \in X^p$ **do**
10: Update position x_i^p, velocity v_i^p using Eq. (16) and (14) respectively.
11: **end for**
12: **if** $F(X^p) > F(\mathfrak{Q}^p)$ **then**
13: Update position, $\mathfrak{Q}^p \leftarrow X^p$
14: **end if**
15: **if** $F(\mathfrak{Q}^p) > F(\mathfrak{g}_{best})$ **then**
16: Update position, $\mathfrak{g}_{best} \leftarrow \mathfrak{Q}^p$
17: **end if**
18: **end for**
19: $Itr = Itr + 1$
20: **end while**
21: **for all** dimension $x_i \in \mathfrak{g}_{best}$ **do**
22: **if** $x_i == 1$ **then**
23: $\mathbb{T} \leftarrow \mathbb{T} \bigcup s_i$
24: **end if**
25: **end for**
26: **return** Desired Team $\mathbb{T}$

In Algorithm 2, we assume a set of particles indicated by P, each of which stands for a solution candidate. The particles search for new positions in the search space by adjusting their velocities in each iteration. The particle's velocity is modified by taking into account the information gathered from its neighbors. The local best position $\mathfrak{L}^p$ represents the particle's individual best solution so far, and the global best position $\mathfrak{g}_{best}$ represents the best solution across all particles in the swarm, which are used to update the particle's velocity. After each iteration, the fitness function F is used to evaluate the quality of the candidate solutions and update the values of $\mathfrak{L}^p$ and $\mathfrak{g}_{best}$. At first, various system parameters are initialized in line 1. We initially generate the random position of the solution vector X^p, and initial velocity, v^p for each particle $p \in P$ using uniform random distribution, in lines 4 and 5, respectively, After that, we update the velocity v_i^p and position x_i^p for each particle $p \in P$ in the search space in line 10. Then, on line 13, we compare the current solution to the particle's local best solution and update the particle's local best. Similarly, on line 16, we compared each particle's local best solution to the global best and updated the global best solution accordingly. We have found our expected solution $\mathfrak{g}_{best}$ once all iterations have been completed. In the last step, on line 23, we convert the expected solution vector $\mathfrak{g}_{best}$ into our desired team $\mathbb{T}$.

4.3.1. Defining the position vector

The position of each particle $p \in P$ is denoted by a k dimensional vector $X^p = (x_1^p, x_2^p, x_3^p, \ldots, x_i^p, \ldots, x_k^p)$, where, each entry $x_i^p \in X^p$ refers to the decision of a worker s_i. If the value of x_i^p is 1, then the worker s_i is selected. Otherwise, it will not be selected. The total number of members in each team must be equal to $\mathcal{N}$, i.e., the number of members in each team must satisfy the following conditions,

$$\sum_{x_i^p \in X^p} x_i^p = \mathcal{N}. \tag{13}$$

At first, we generate the initial position vector X^p of each particle using uniform random distribution, where, $x_i^p \sim U(b_l, b_u)$. The values b_l and b_u represent the lower and upper boundaries of the search space, respectively.

4.3.2. Updating particle position and velocity

At first, for each particle $p \in P$, the initial velocity, v_i^p is bounded by $U(-|b_l, b_u|, |b_l, b_u|)$. Then the velocity of it's i^{th} component, $v_i^p(t+1)$ is updated according to the following equation:

$$v_i^p(t+1) = v_i^p(t) + z_1 \times r_1 \times (\mathfrak{L}_i^p(t+1) - x_i(t)) + z_2 \times r_2 \times (\mathfrak{g}_{best}(t+1) - x_i(t)), \tag{14}$$

where, r_1 and r_2 are two random values those are uniformly distributed between [0, 1] and t indicates the tth iteration number. Here, $\mathfrak{L}_i^p$ and $\mathfrak{g}_{best}$ denote the local best and the global best, respectively of the ith component of particle $p \in P$. The constants z_1 and z_2 represent the cognitive and social components, respectively, which determine the extent to which the particles are influenced by their personal best and global best solutions.

Here, the value of $v_i^p(t+1)$ calculated from Eq. (14) provides us a continues value. We need to convert the values into binary variables to avoid fractional values. The sigmoid function is used to convert the velocity entry into a probability value using Eq. (15) as follows,

$$\sigma(v_i^p(t+1)) = \frac{1}{1 + e^{-v_i^p(t+1)}}, \tag{15}$$

$$x_i^p(t+1) = \begin{cases} 1, & \sigma(v_i^p(t+1)) \geqslant rand() \\ 0, & \text{otherwise.} \end{cases} \tag{16}$$

Then, using Eq. (16), we have updated the position of each particle $p \in P$. Here, $rand()$ is a random value within [0, 1]. If the value of the sigmoid function suppresses the $rand()$ function, then we have added the worker $s_i \in S$ in the desired team $\mathbb{T}$.

4.3.3. Fitness function

To update the local best solution $\mathfrak{L}^p$ of a particle $p \in P$ and the global best value $\mathfrak{g}_{best}$, we need to define a fitness function $F(.)$, which calculates the fitness value of a solution X^p. The fitness of a solution X^p is defined as follows,

$$F(X^p) = \begin{cases} \sum_{x_i \in X^p} \Gamma_1 \times \mathscr{C}_i + \Gamma_2 \times \Omega_{M,i} - \Gamma_3 \times \bar{t}_i - \Gamma_4 \times \bar{c}_i, \\ \text{If Eq. (13) is satisfied} \\ -1, \qquad \text{Otherwise.} \end{cases} \tag{17}$$

Here, Eq. (17) is the objective function used to summarize the solution for a nearly optimal team finding problem as a single figure of merit. Here, $\Gamma_1, \Gamma_2, \Gamma_3$, and Γ_4 are the weight factors and $\Gamma_1 + \Gamma_2 + \Gamma_3 + \Gamma_4 = 1$. Depending on the buyer's preferences, the weight factors are assigned accordingly. Then, based on the fitness function $F(.)$ value, the value of the local best $\mathfrak{L}^p$ and global best, $\mathfrak{g}_{best}$ solution in lines 14 to 17 of Algorithm 2, are updated. Then finally, the PCS system can form the desired team $\mathbb{T}$.

Here, in Eq. (17), $\bar{t}_i, \bar{c}_i, \mathscr{C}_i, \Omega_{M,i}$ denote the normalized required task completion time, normalized worker's claimed cost, worker's technical competency, and worker's team work capacity, respectively, range in [0,1]. The normalized cost and task execution time using Eqs. (18) and (19), respectively can be determined as follows,

$$\bar{c}_i = \frac{c_i}{C}, \tag{18}$$

$$\bar{t}_i = \frac{t_i}{T_j^d}. \tag{19}$$

4.4. Worker payment policy

After getting the results from the platform, the buyers send the claimed price to the platform, and then the platform distributes the payment to the workers. The developed PCS system can adopt the worker payment policy similar to (Roy et al., 2021), which rewards workers for providing high-quality service while penalizing them for failing to meet task quality requirements. A reward policy like this encourages workers to participate in the platform and make significant contributions. The platform evaluates the quality of the task. We use a metric, q_i similar to (Sarker et al., 2019), namely task quality score, which assesses the worker's task quality and can be determined as follows,

$$q_i = \Pi_i - \Pi_{i*}, \tag{20}$$

where, Π_i and Π_{i*} denote the assigned task quality and actual task quality. where, Π_i and Π_{i*} denote the given task quality and true task quality. According to PCS policy, payment ρ_i of a worker, $s_i \in S$ for completing a buyer's task can be evaluated as follows,

$$\rho_i = \begin{cases} c_i + \varphi, & q_i \leqslant q_{max} \&\& t_i < T_j^d, \\ \epsilon \times c_i, & q_i > q_{max}, \\ c_i, & otherwise. \end{cases} \tag{21}$$

Here, q_{max} denotes the maximum allowable task quality variance and t_i denotes the task completion time. The values of q_{max} and T_j^d are calculated by the nature of the applications, which are determined by the buyer. In the case of $q_i \leqslant q_{max}$ and $t_i < T_j^d$, the worker s_i delivers the buyer b_j with a high task quality as well as a high QoE and φ amount of incentive is added to the claimed price of the worker. On the other hand, $q_i > q_{max}$ indicates that the worker s_i provides poor task quality that penalizes the worker by a factor of $\epsilon \in [0, 1]$. Otherwise, the worker is given the claimed cost of the task. Different platforms adopt different strategies for calculating the incentive amount φ and penalty amount ϵ. In the developed PCS system, the incentive amount, φ can be determined using the method similar to (Roy et al., 2021) as follows,

$$\varphi = (\mathscr{V}_i - c_i) \times \{1 - e^{(q_i - q_{max}) \times \theta_1}\}, \tag{22}$$

where, $\mathscr{V}_i$ is market value of task which is calculated by platform. The penalty factor ϵ can be determined using Eq. (23) as follows,

$$\epsilon = e^{(q_i - q_{max}) \times \theta_2}. \tag{23}$$

Here, θ_1 and θ_2 are the amplification factors that amplify the effect of a task's quality departure q_i from the maximum allowable variation. The Higher values of θ_2 and θ_1 have negative effects on the workers' claimed price in terms of penalties and positive effects in terms of rewards, respectively. Finally, buyers use Eq. (1)–(12) to update the workers' reputations based on their interaction experience.

4.5. Complexity analysis

In this section, we analyze the complexity of the developed PCS system. At first, we analyze the complexity of Algorithm 1. In the worst case, short-listing workers in statement 2 requires checking M workers, resulting in computational complexity of $O(M)$. In statement 9, we use merge sort, which has a worst-case computational complexity of $O(M \times log(M))$. The other lines are constant time complexity. As a result, Algorithm 1's total time complexity is $O(M + M \times log(M)) \approx O(M \times log(M))$. Next, we analyze the complexity of Algorithm 2. In the worst case, statements 3–6 are enclosed in a loop that iterates P times, resulting in computational complexity of $O(P)$. Similarly, the computational complexity of statements 9–11 is $O(|\mathbb{X}^p|)$. Let I denote the maximum number of iterations. Statements 7–20 are enclosed in a while loop having a worst-case computational complexity of $O(I \times P \times |\mathbb{X}^p|)$. Finally, Lines 21 to 25 are enclosed in a loop that iterates $|\mathbb{X}^p|$ time. As a result, Algorithm 2's total time complexity is $O(P + (I \times P \times |\mathbb{X}^p|) + |\mathbb{X}^p|) \approx O(I \times P \times |\mathbb{X}^p|)$.

5. Performance evaluation

In this section, we have implemented the PCS system using MATLAB and discussed on its comparative performances with the state-of-the-art works- the Greedy system (Wang et al., 2019), and the STAM (Yu et al., 2019).

5.1. Simulation environment

We assume that software development tasks and workers are randomly distributed on an online platform. We employ a dataset from the well-known web-based crowdsourcing platform Fiverr to validate the suggested techniques under realistic conditions (Freelancers' offers on fiverr, 2023). We have gathered information about 500 workers' competencies and payment histories and created dynamic teams in accordance. Work time and skill-level scores are among the details provided by the employees. The task data for the experiment was obtained using historical information, numerous platform investigations, and computer modeling. Each worker's per unit resource bidding price is chosen using a uniform random distribution from the range of USD 10 to 30 and the buyer's budget is chosen from the range of USD 10 to 35. We have categorized 130 tasks related to programming and tech, like web programming, user testing, UX/UI designing, and so on. We have considered the development of software applications having a deadline range between 3 and 15 days. In comparison, task completion time is 1 to 20 days, e.g., the worker can submit their tasks in this time frame, but work completed after the deadline will result in a penalty. The above settings of the parameter values represent the development of practical software applications, as depicted in Xie and Lui (2018); Shen et al., 2022. Each experiment is run for 1000 s, and the average outcomes of 30 simulations run using various random seed values are used to plot each data point on the graph. The performance parameters employed in multiple studies are all listed in Table 2 along with their respective ranges. All simulation trials are run on a desktop computer having an Intel (R) Core(TM) i5-7200U CPU of 2.80 GHz and 8 GB memory.

5.2. Performance metrics

On the following metrics, we evaluated the proposed PCS system's performance.

- **Average utility of buyers** indicates the utility a buyer receives from a dependable worker for doing a task successfully. A higher score implies that the buyer is more pleased with the task's final result. The buyer's utility $\mathscr{U}_j$ calculation has been done by trading off between the cost c_i and task quality q_i.

$$\mathscr{U}_j = \varphi \times c_i + (1 - \varphi) \times q_i \tag{24}$$

- **Quality-of-Experience (QoE)** indicates the inverse relationship between the task completion time and task deadline, which can be determined as follows,

$$QoE = 1 - \frac{t_i}{T_j^d}. \tag{25}$$

It allows us to track how fast a buyer receives a task from a team.

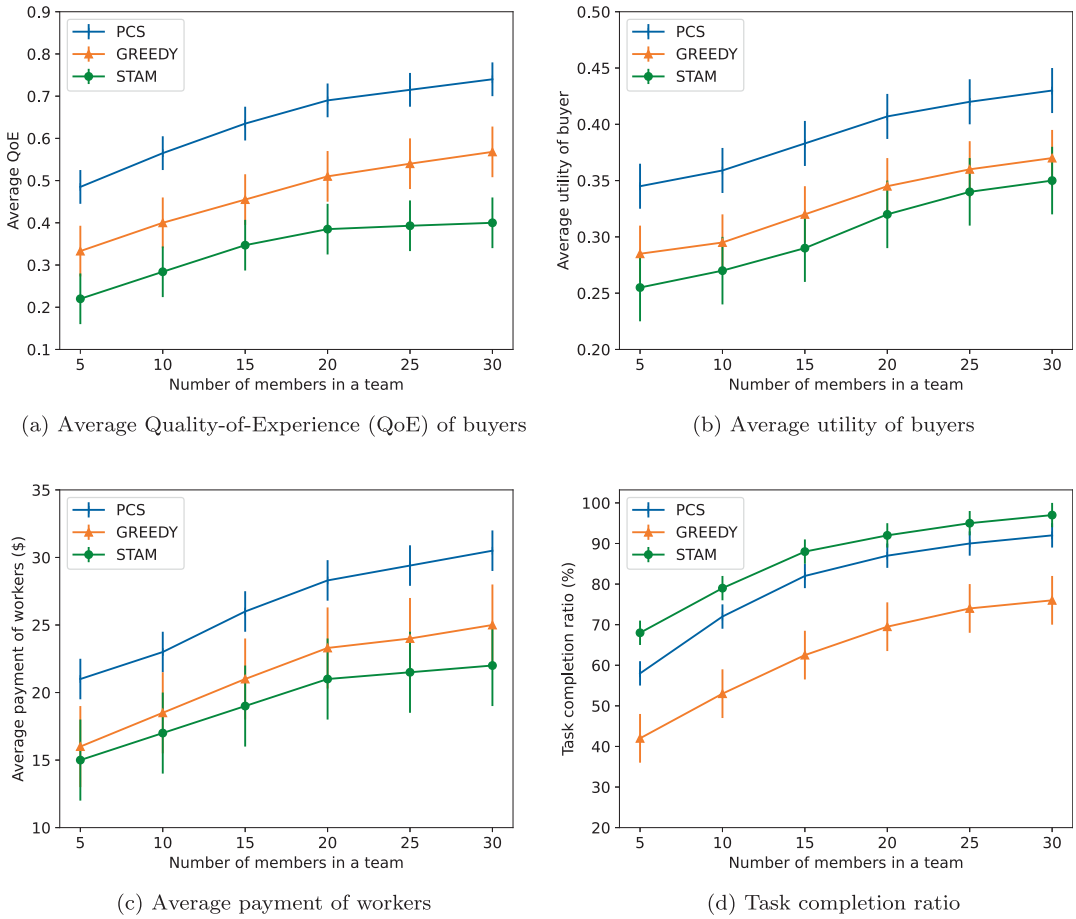
- **Task completion ratio** specifies the number of the buyers' trades that were successfully completed.
- **Average payment of workers** denotes the amount of payment received by a worker after task completion.

5.3. Simulation result

In this section, we have evaluated the performance of the proposed PCS model by varying the number of members in a team, task diversity, task delay deadline, and task budget. Also, we have

Table 2
Simulation parameters.

Parameters	Value
Total number of platform workers	500
Number of members in a team	$5 \sim 30$
Diversity of tasks	$3 \sim 18$
Worker's bid in dollar ($) per unit task	$10 \sim 30$
Buyer's budget in dollar ($) per unit task	$10 \sim 35$
Task deadline	$3 \sim 15$ days
Task completion time	$1 \sim 20$ days
Simulation time	1000 s
Weight parameters	$\alpha, \kappa, \zeta = 0.7$
Maximum Iteration, MAX-IT	200



(a) Average Quality-of-Experience (QoE) of buyers

(b) Average utility of buyers

(c) Average payment of workers

(d) Task completion ratio

Fig. 3. Impacts of varying number of members in a team.

presented the impact of the developed reputation scheme for worker selection.

5.3.1. Impact of varying number of members in a team

In this experiment, we vary the maximum number of members in a team by keeping the task diversity and task delay deadline fixed at 10 and 12 days, respectively. The team size increases with the joining of more dependable members in the team, which improves the system performance displayed in Fig. 3.

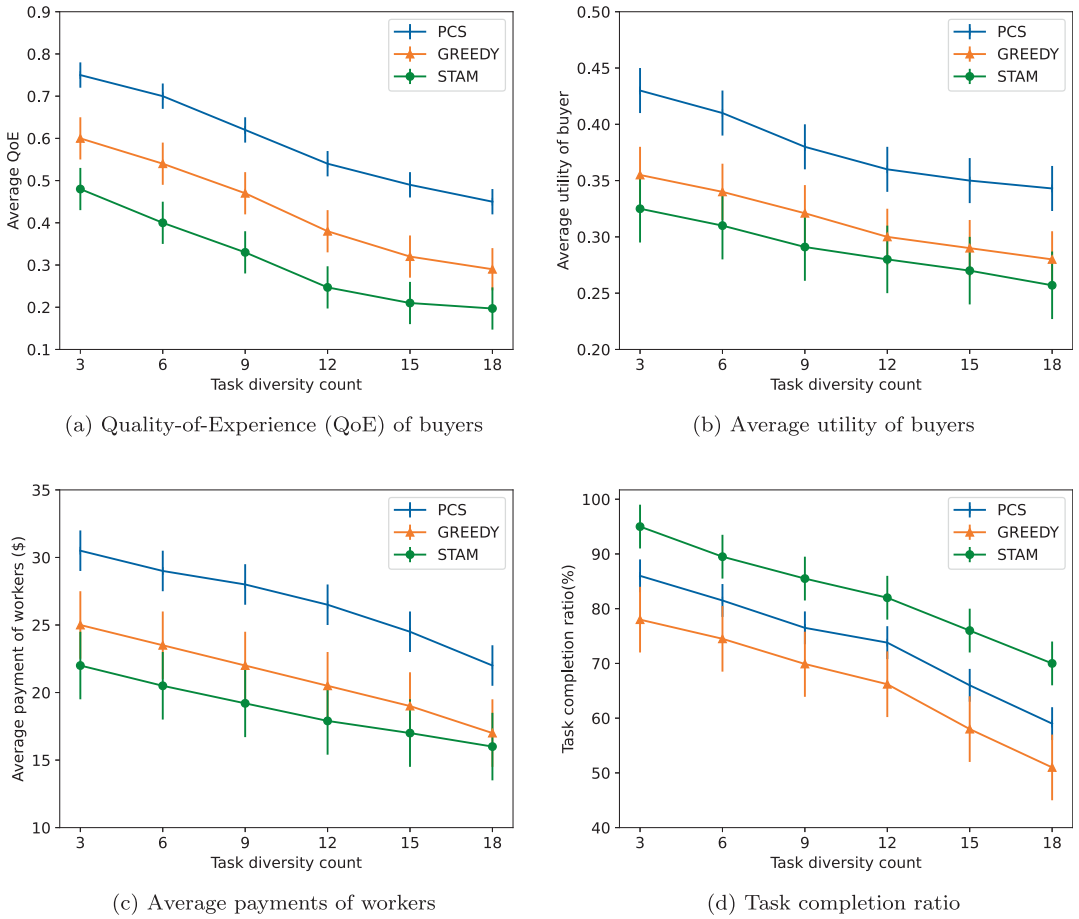
With the increasing number of members in a team, buyers' QoE is increased as illustrated in Fig. 3(a). This is because more workers are available, giving the buyers a range of trustworthy options to select from. Additionally, since the number of tasks is static, buyers can receive service quickly, which raises the QoE. The STAM system allocates the task to a worker by considering the worker's active time and task module complexity. On the other hand, the greedy technique only considers the worker's mutual distance, skill, and cost, and they haven't considered a worker's reputation, which supports their prior history. As a result, PCS excels in terms of QoE. For a similar reason, the PCS performs better in terms of buyer's utility because the PCS system considers reputation and QoE in the calculation, which is shown in Fig. 3(b).

With an increase number of members in a team, the average payment of workers improves, as seen in Fig. 3(c). This is because, with the increasing number of member in a team, the likelihood of choosing more trustworthy workers enables buyers and workers to receive high-quality tasks and rewards for successfully completing them. High-quality workers can give services with less complexity and deliver them to customers before the task deadline because there are limited types of tasks to do. The Greedy system uses a threshold payment mechanism, penalizing workers on task completion that does not constitute a task deadline. Additionally, neither the Greedy system nor the STAM offers rewards to the workers for delivering greater QoE and successfully completing tasks. As a result, the PCS system excels in terms of the average payment of workers.

From Fig. 3(d), we observe that the task completion ratio appears to be rising as the number of members in a team increases. The rationale is that more workers equate to more resources at their disposal to complete the task. The Greedy system performs worst in such a situation because their method of choosing team work capacity is based on how well-connected the workers are in their social networks, which limits the search, reduces the likelihood of finding additional workers, and lowers task completion rates. On the other hand, the STAM works best because it only assigns workers after evaluating their active time and necessary skill sets. This leads to quick worker allocation but raises questions about hiring honest workers. The proposed PCS system builds teams by considering workers' technical competency and prior performance, ensuring timely delivery of high-quality tasks and ideal team building.

5.3.2. Impact of task diversity

In this section, we have presented the outcomes of the developed PCS model's performance by varying the number of task diversity, i.e., types of categories of competency required to complete a project, e.g., web programming, user testing, UI/UX designing, and so on. In this experiment, the number of members in a



(a) Quality-of-Experience (QoE) of buyers

(b) Average utility of buyers

(c) Average payments of workers

(d) Task completion ratio

Fig. 4. Impacts of varying task diversity.

team and the task delay deadline are kept fixed at 15 and 12 days, respectively.

Fig. 4(a) demonstrates that when the task diversity increases, the QoE of the buyers is lowered. The justification is that there need to be more workers to manage the expanding variety of tasks, which results in buyers receiving service slowly. However, the PCS system outperforms the STAM and the Greedy method because they have assigned a task to a worker by considering their available time with the requisite skill set and ignoring their prior work history. In contrast, the PCS develops the worker's prior performance knowledge using EWMA, which makes the procedure more sensitive.

The average utility of each buyer decreases as the task diversity increases, as seen in Fig. 4(b). This is because more task diversity means more skill sets must be satisfied and more workers must be hired, and many buyers are unable to choose trustworthy workers from whom a buyer can obtain a greater utility due to the system's limited supply of workers. However, the PCS system outperforms the Greedy and the STAM since they don't take into account a worker's reputation or the cost-quality trade-off when making the final hiring decision.

Average payment of workers for task completion is reduced with the increasing number of task diversity for all evaluated systems is depicted in Fig. 4(c). Here, we see that the average payment decreases as the task diversity increases. However, compared to the Greedy and the STAM system, the payment in the suggested PCS system is more significant because the PCS system rewards workers for producing work of greater quality and penalizes them for completing tasks of lower quality or for incompleteness.

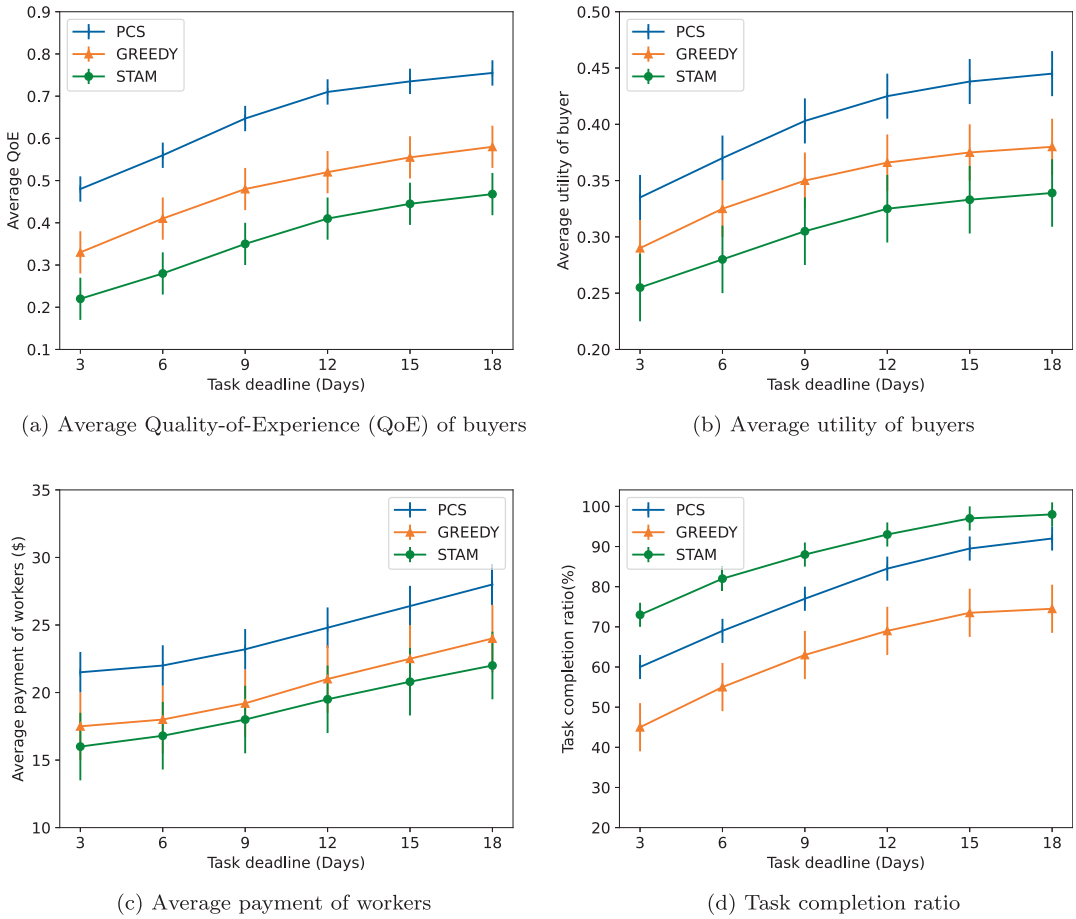
As per Fig. 4(d), the task completion ratio decreases as the quantity of task diversity increase. Some tasks cannot be assigned to the workers with the increasing task diversity because of the system's fixed worker number in a team. The STAM system performs best in this case because it only allocates workers after verifying their active time and requisite skill set. This leads to rapid worker allocation, but sometimes it fails to hire the most qualified and truthful workers. On the other hand, the Greedy system only works successfully when the workers are closely connected within a social network; otherwise, it is unable to match the right worker with the task.

5.3.3. Impact of varying deadline of task

In this section, we have presented the outcomes of the suggested PCS model's performance by varying the task deadline and keeping the number of members in a team fixed at 15 and task diversity set at 10.

Fig. 5(a) shows that when the task deadline increases, the buyer's QoE increases. This is because when the task delay deadline is higher for task completion, workers feel less stressed about the work and produce more creative and quality work within the task delay deadline. The PCS performs better by selecting the highly truthful workers by considering the prior performance of them. For the same reason, the average utility of the buyer is increased with the increasing deadline, which is shown in Fig. 5(b). This is because, with the increasing task delay deadline, workers can get enough time to complete the task with high quality. The PCS system outperforms the STAM and Greedy systems because of the worker selection system.

Fig. 5(c) demonstrates that the payment is decreasing with the increase of task deadline. This happens because, when time increases with the same task diversity amount and team members,



(a) Average Quality-of-Experience (QoE) of buyers

(b) Average utility of buyers

(c) Average payment of workers

(d) Task completion ratio

Fig. 5. Impacts of varying deadline of tasks.

team members get more flexibility to complete their tasks by maintaining quality. Unlike the STAM and the greedy system, PCS is giving rewards to workers, and PCS workers are getting better payment. The task completion ratio is also increased as the task deadline increases, which is illustrated in Fig. 5(d). The STAM functions best since it only assigns employees after confirming their active time and necessary skill set. Rapid worker allocation results from this, raising concerns about employing honest personnel. The Greedy system, on the other hand, can only match the right worker to the right job if the workers are closely linked in a social network. For that reason, the PCS system outperforms the state-of-the-art works.

5.3.4. Impact of varying budget of buyer

In this section, we have presented the outcomes of the suggested PCS model's performance by varying the budget of the buyer and keeping the number of members in a team fixed at 15 and task diversity set at 10.

Fig. 6(a) shows that when the buyer's budget increases, his QoE increases. This is because when the task's budget is higher it attracts higher-quality workers who give quality task output. Also, with a higher budget, the buyer can provide more rewards to the workers, which leads to them completing tasks quickly and increases the buyer's utility as in Fig. 6(b). The PCS system gives a better result than the STAM and Greedy system because, in the PCS system, both worker selection and team formation focuses on the cost and quality trade-off that results in selecting a highly skilled worker team.

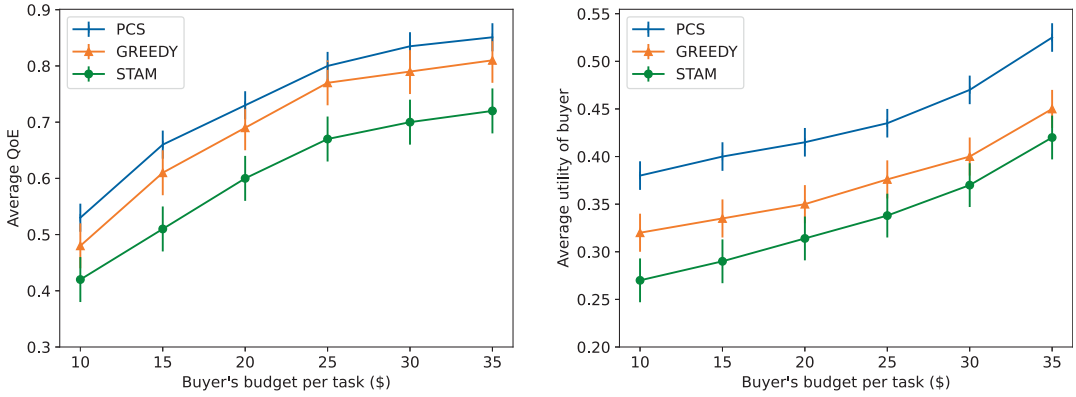
The worker's payment is improved with the increase of the buyer's budget, which is demonstrated in Fig. 6(c). As a buyer increase the task budget, more skillful workers get selected from the system which results in automatically higher payment but in the PCS system, the worker gets rewarded along with the payment if they give a timely task delivery. The PCS system gives better results than the STAM and Greedy system because of its reputation-aware worker selection, robust team formation, and reward system for deserving workers.

From Fig. 6(d), we observe that the task completion ratio is rising as the buyer's budget increases because with a rising budget worker's motivation increase to work and results in quality task completion within the time constrain. The STAM results best because of its rapid worker allocation technique. Although it raises concerns about employing honest personnel. The Greedy system's success is contingent on workers being only closely connected within a social network. The PCS system on the other hand selects reliable workers and an efficient worker team and gives a decent successful task completion rate.

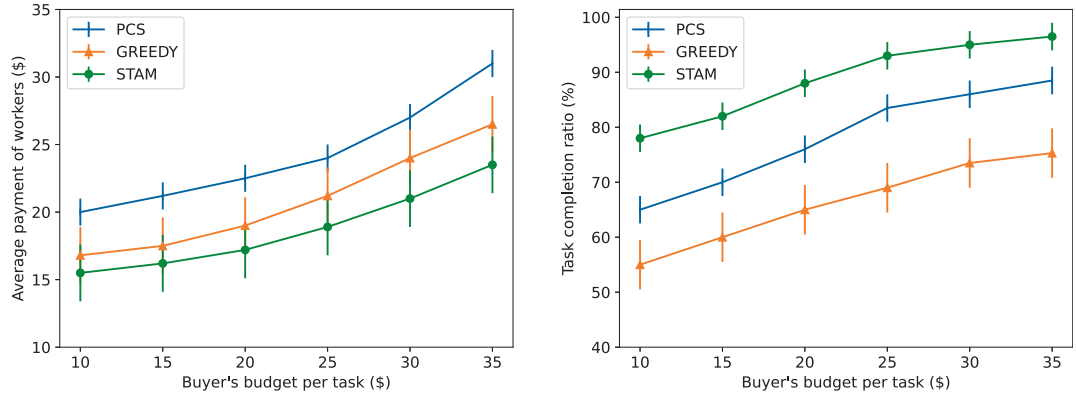
5.3.5. Impact of worker reputation scheme

In this experiment, we have analyzed the effects of the developed reputation scheme for worker selection in the PCS system by varying the task deadline from stringent to relaxed.

Fig. 7(a) shows that the average reputation value of the workers decreases with the diverse variety of tasks. This is due to the fact that with the diverse variety of tasks, workers require a diverse level of technical skill and it requires higher time to complete tasks within the deadline, which in turn reduces the reputation of the workers. From Fig. 7(b) we see that in the PCS system, the successful detection rate of unreliable workers increases as the reputation threshold increases. When the deadline is 18 days and the reputa-

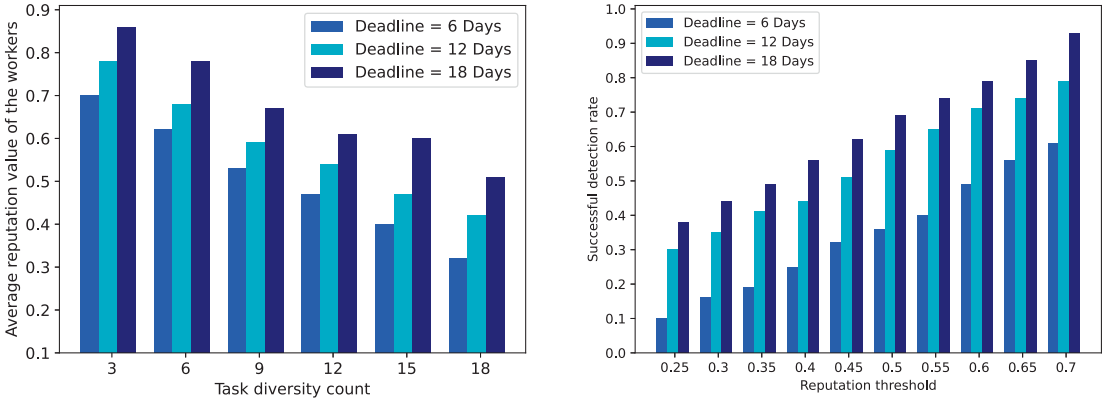


(a) Average Quality-of-Experience (QoE) of buyers (b) Average utility of buyers



(c) Average payment of workers (d) Task completion ratio

Fig. 6. Impacts of varying budgets of a buyer.



(a) Reputation value of workers (b) Successful detection ratio

Fig. 7. Impact of worker reputation scheme.

tion threshold is 0.7, the successful detection rate is almost 90%, while with the same reputation threshold but with a more stringent deadline the detection rate falls significantly. It shows that for getting a more reliable worker team, relaxed task deadlines can play an important role.

6. Conclusion and future work

This paper introduced a general framework, namely PCS, for hiring workers and making an optimal team in collaborative software crowdsourcing for the smart city. By keeping Industry 5.0 in perspective, we focused on making the developed PCS system as intelligent, personalized, and automated as possible, as well as human-centric, resilient, and sustainable. The PCS system can be successfully implemented in any online freelancing platform and software company as they involve a wider community of developers and experts working remotely on a project. By exploiting the PCS system, new offspring software companies can also build a trustworthy, sustainable, and collaborative work environment. Our reputation calculation based on the worker's technical competency

and prior performance helped to select the most truthful workers. The optimal team was formed by exploiting the meta-heuristic AI-based particle swarm optimization technique. Our developed payment mechanism provided the incentive to the workers along with regular payments which motivated the workers to provide high-quality tasks as well as penalized the worker for the poor quality task. The simulation results carried out in MATLAB depicted that the developed PCS system improved the Quality-of-Experience (QoE) and utility as high as 30% and 25%, respectively, compared to the state-of-the-art works.

Determining the team size and task diversity considering multiple conflicting parameters following a scientific theory can be an interesting research problem. Meta-heuristic methods other than PSO can be investigated for testing their suitability in different scenarios. Moreover, exploiting deep learning approaches to reward mechanisms would be a fascinating area for further research, which has been kept on our list of future research.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors are grateful to King Saud University, Riyadh, Saudi Arabia for funding this work through Researchers Supporting Project Number (RSP2023R18). This work was also supported by the Center for Research, Innovation, and Transformation (CRIT) of Green University of Bangladesh.

References

Chakir, O., Rehaimi, A., Sadqi, Y., Krichen, M., Gaba, G.S., Gurtov, A., et al., 2023. An empirical assessment of ensemble methods and traditional machine learning techniques for web-based attack detection in industry 5.0. J. King Saud Univ.-Comput. Informat. Sci. 35 (3), 103–119.

Duan, Y., Harley, R., Habetler, T., 2009. Comparison of particle swarm optimization and genetic algorithm in the design of permanent magnet motors. In: 2009 IEEE 6th International Power Electronics and Motion Control Conference, pp. 822–825.

Dubey, A., Abhinav, K., Taneja, S., Virdi, G., Dwarakanath, A., Kass, A., Kuriakose, M.S, 2016. Dynamics of software development crowdsourcing. In: 2016 IEEE 11th International Conference on Global Software Engineering (ICGSE), pp. 49–58.

E. Commission, D.-G. for Research, Innovation, Breque, M., De Nul, L., Petridis, A., 2021. Industry 5.0: towards a sustainable, human-centric and resilient European industry. Publications Office.

Estrada, R., Valeriano, I., Torres, D., 2022. Multi-task versus consecutive task allocation with tasks clustering for mobile crowd sensing systems. Proc. Comput. Sci. 198, 67–76, 12th International Conference on Emerging Ubiquitous Systems and Pervasive Networks/ 11th International Conference on Current and Future Trends of Information and Communication Technologies in Healthcare.

Freelancers' offers on fiverr. https://www.kaggle.com/datasets/kirilspiridonov/freelancers-offers-on-fiverr. [Online; accessed 12-March-2023].

Fu, H.C.Y., Song, F., 2015. STWM: A solution to self-adaptive task-worker matching in software crowdsourcing, p. 383–398.

Gu, Q., Liu, Y., Chen, L., Xiong, N., 2022. An improved competitive particle swarm optimization for many-objective optimization problems. Expert Syst. Appl. 189, 116118.

Tommaselli G.A,, Sehat A.J., Ricketts C.D., Clymer J.W. and Grange Ph., 2022. Value of the crowd-sourced assessment of technical skills (c-sats) platform in surgical procedures: A systematic review of evidence. Surg Res. 4(2), 1–12," Correspondence: Jeffrey W. Clymer, Ethicon Inc, vol. 4545.

Guo, Y., Wang, H., He, L., Pedrycz, W., Suganthan, P.N., Song, Y., 2023. A reinforcement learning-assisted genetic programming algorithm for team formation problem considering person-job matching.

Gutiérrez, J.H., Astudillo, C.A., Ballesteros-Pérez, P., Mora-Melià, D., Candia-Véjar, A., 2016. The multiple team formation problem using sociometry. Comput. Oper. Res. 75, 150–162.

Hamrouni, A., Ghazzai, H., Alelyani, T., Massoud, Y., 2022. Low-complexity recruitment for collaborative mobile crowdsourcing using graph neural networks. IEEE Internet Things J. 9 (1), 813–829.

Haqbani, O.A., Alyahya, S., 2022. Supporting coordination among participants in crowdsourcing software design. In: 2022 IEEE/ACIS 20th International Conference on Software Engineering Research, Management and Applications (SERA), pp. 132–139.

Heling, C., Fei, W., Miaolei, D., Xianyong, W., Yonghe, C., 2023. Software multiple-fault localization using particle swarm optimization via genetic operation. J. King Saud Univ.-Comput. Infr. Sci.

Huang, Y., Chen, M., 2019. Improve reputation evaluation of crowdsourcing participants using multidimensional index and machine learning techniques. IEEE Access 7, 118055–118067.

Huq, F., Sultana, N., Sarkar, S., Razzaque, M.A., Kamal Tushar, M.H., 2019. Optimal worker selection for maximizing quality-of-service of online food delivery system. In: 2019 International Conference on Sustainable Technologies for Industry 4.0 (STI), pp. 1–6.

Jiang, J., An, B., Jiang, Y., Lin, D., 2020. Context-aware reliable crowdsourcing in social networks. IEEE Transa. Syst. Man Cybernet.: Syst. 50 (2), 617–632.

Jiang, J., Di, K., An, B., Jiang, Y., Bu, Z., Cao, J., 2022. Batch crowdsourcing for complex tasks based on distributed team formation in e-markets. IEEE Trans. Parallel Distrib. Syst. 33 (12), 3600–3615.

Liao, Z., Xu, X., Fan, X., Zhang, Y., Yu, S., 2021. Grbmc: An effective crowdsourcing recommendation for workers groups. Expert Syst. Appl. 179, 115039.

Li, J., Zhu, Y., Hua, Y., Yu, J., 2015. Crowdsourcing sensing to smartphones: A randomized auction approach. In: 2015 IEEE 23rd International Symposium on Quality of Service (IWQoS), pp. 219–224.

Machado, L.S., Melo, R.R.M., de Souza, C.R.B., Prikladnicki, R., 2021. Collaborative behavior and winning challenges in competitive software crowdsourcing. In: Proc. ACM Hum.-Comput. Interact., vol. 5, no. GROUP, Jul 2021. [Online]. Available: https://doi.org/10.1145/3463932.

Maddikunta, P.K.R., Pham, Q.-V., B, P., Deepa, N., Dev, K., Gadekallu, T.R., Ruby, R., Liyanage, M., 2022. Industry 5.0: A survey on enabling technologies and potential applications. J. Indu. Infr. Integrat. 26, 100257.

Mao, K., Yang, Y., Wang, Q., Jia, Y., Harman, M., 2015. Developer recommendation for crowdsourced software development tasks. In: 2015 IEEE Symposium on Service-Oriented System Engineering. IEEE, pp. 347–356.

Rahman, S.T., Sinthia, A.K., Akter, S.N., Roy, P., Razzaque, M.A., 2021. Reputation aware fair worker selection in collaborative software crowdsourcing. In: 2021 3rd International Conference on Sustainable Technologies for Industry 4.0 (STI). Dhaka, Bangladesh: IEEE, pp. 1–6.

Routledge, R., 1979. Diversity indices: Which ones are admissible? J. Theor. Biol. 76 (4), 503–515.

Roy, P., Sarker, S., Razzaque, M.A., Hassan, M.M., AlQahtani, S.A., Aloi, G., Fortino, G., 2020. AI-enabled mobile multimedia service instance placement scheme in mobile edge computing. Comput. Netw. 182, 107573.

Roy, P., Sarker, S., Razzaque, M.A., or Rashid, M.M., Hassan, M.M., Fortino, G., 2021. Distributed task allocation in mobile device cloud exploiting federated learning and subjective logic. J. Syst. Architect. 113, 101972, 2021.

Sarker, S., Razzaque, M.A., Hassan, M.M., Almogren, A., Fortino, G., Zhou, M., 2019. Optimal selection of crowdsourcing workers balancing their utilities and platform profit. IEEE Internet Things J. 6 (5), 8602–8614.

Shen, S., Ji, M., Wu, Z., Yang, X., 2022. An optimization approach for worker selection in crowdsourcing systems. Comput. Ind. Eng. 173, 108730.

Sorour, S.E., Abdelkader, H.E., Sallam, K.M., Chakrabortty, R.K., Ryan, M.J., Abohany, A., 2022. An analytical code quality methodology using latent dirichlet allocation and convolutional neural networks. J. King Saud Univ.-Comput. Infr. Sci. 34 (8), 5979–5997.

Souza-Pereira, L., Pombo, N., Ouhbi, S., 2022. Software quality: Application of a process model for quality-in-use assessment. J. King Saud Univ.-Comput. Infr. Sci. 34 (7), 4626–4634.

Stol, K.-J., LaToza, T.D., Bird, C., 2017. Crowdsourcing for software engineering. IEEE Software 34 (2), 30–36.

Sun, S., Sun, L., Ma, X., Pan, Z., Jin, H., 2022. A trusted distributed crowdsourcing framework based on user preferences. In: 2022 IEEE International Performance, Computing, and Communications Conference (IPCCC), pp. 43–48.

Wagner, G., Prester, J., Paré, G., 2021. Exploring the boundaries and processes of digital platforms for knowledge work: A review of information systems research. J. Strateg. Inf. Syst. 30, (4) 101694.

Wang, W., Jiang, J., An, B., Jiang, Y., Chen, B., 2017a. Toward efficient team formation for crowdsourcing in noncooperative social networks. IEEE Trans. Cybernet. 47 (12), 4208–4222.

Wang, Z., Sun, H., Fu, Y., Ye, L., 2017b. Recommending crowdsourced software developers in consideration of skill improvement. In: 2017 2nd IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, pp. 717–722.

Wang, W., He, Z., Shi, P., Wu, W., Jiang, Y., An, B., Hao, Z., Chen, B., 2019. Strategic social team crowdsourcing: Forming a team of truthful workers for crowdsourcing in social networks. IEEE Trans. Mob. Comput. 18 (6), 1419–1432.

Wang, L., Yang, D., Yu, Z., Han, Q., Wang, E., Zhou, K., Guo, B., 2021. Acceptance-aware mobile crowdsourcing worker recruitment in social networks. IEEE Trans. Mob. Comput. 1–1.

Wang, H., Li, J., Song, Y., Huang, J., Li, J., Chen, Y., 2022. An improved genetic algorithm for team formation problem. In: 2022 IEEE Symposium Series on Computational Intelligence (SSCI), pp. 774–781.

Xiao, F., Wang, H., Guo, S., Guan, X., Liu, B., 2021. Efficient and truthful multi-attribute auctions for crowdsourced delivery. Int. J. Prod. Econ. 240, 108233.

Xie, H., Lui, J.C.S., 2018. Incentive mechanism and rating system design for crowdsourcing systems: Analysis, tradeoffs and inference. IEEE Trans. Serv. Comput. 11 (1), 90–102.

Yu, D., Zhou, Z., Wang, Y., 2019. Crowdsourcing software task assignment method for collaborative development. IEEE Access 7, 35743–35754.

Zhang, Y., Qin, H., Li, B., Wang, J., Lee, S., Huang, Z., 2018. Truthful mechanism for crowdsourcing task assignment. Tsinghua Sci. Technol. 23 (6), 645–659.